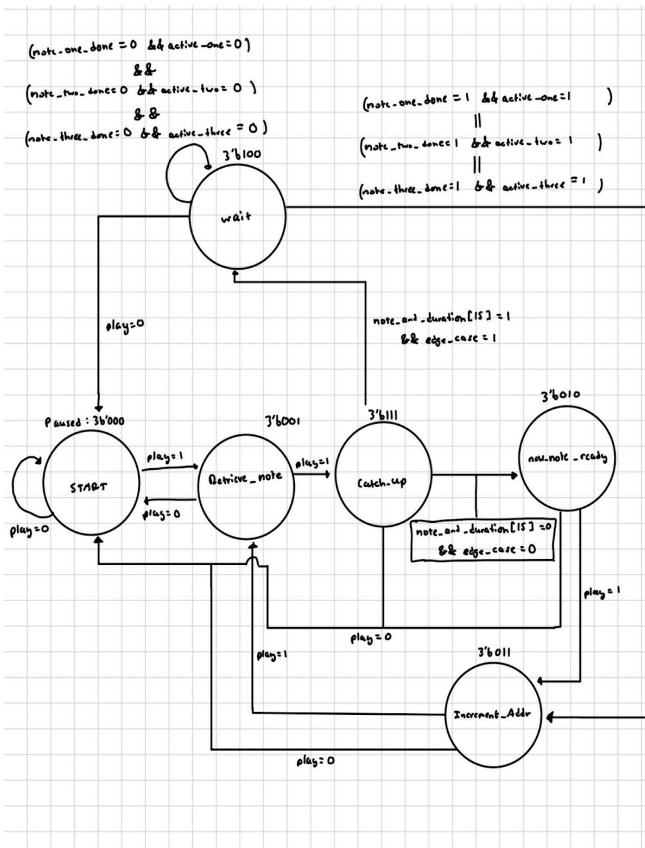


EE 108 Final Report

Our music synthesizer implemented the following extensions: Chords (expected 4 points), PMOD (expected 4 points), Pulse width modulation (expected 1 point), harmonics (expected 3 points), and playback control (expected 1-2 points). Below we include the specification, usage, high-level design, key implementation details, and challenges we faced for each extension in their respective section.

Chords Specification, Usage, High-Level Design, Key-Implementation, and Challenges

Our chord extension enabled us to schedule and play up to 3 notes simultaneously. To achieve this, we had to redesign our song_reader and note_player in a way that allowed us to schedule 1 to 3 notes and then play them for a certain amount of time. The most straightforward solution we found was to instantiate three instances of note_player in music_player and then schedule as



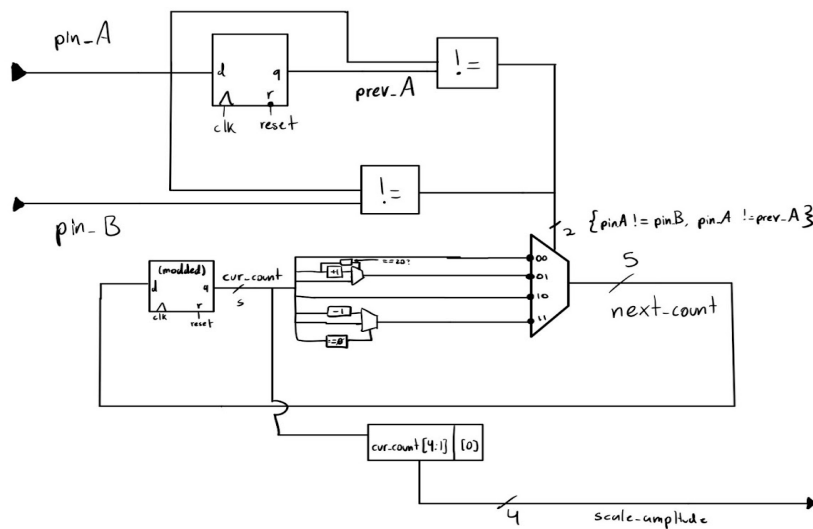
many notes as our chords in our SONG_ROM indicated. This was achieved by reformulating our states for song_reader to allow us to cycle and schedule all notes in a chord as long as the MSB of the SONG_ROM output is 0. Note that the path for the cycle in which we're scheduling notes consists of retrieving the note, catching up (buffer state to account for the SONG_ROM's one-cycle delay), new note ready, increment address, retrieving a new note, and then waiting state where it decides if more notes need to be scheduled or if the schedules notes are okay to be played. In the instance where the SONG_ROM MSB output becomes one, the song_reader enters the wait state and plays the previously scheduled notes for the current advance time duration. As soon as the advance time duration is finished, we schedule as many new notes

as we can/have to and enter the wait state again. It's important to note that whenever we enter the new_note_ready state, one of three one-pulsed signals is triggered to tell its respective note_player instantiation in music_player to latch in the current note and duration. The hardest obstacle that we encountered when implementing Chords was being able to find a methodical way to only have one note and note_duration output and have it be stored by the note_player of

our choice (or the note player whose note was finished). As mentioned before, we overcome this obstacle by simply instantiating three new `_note_ready` signals (one for each `note_player`). Whenever a new `_note_ready` signal is one-pulsed, its respective `note_player` would use it as an enable bit for its `note` and `note_duration` flip flops to know when to store new values. For testing, we made sure to load multiple notes at once and had a counter to ensure we counted up to the duration value with new notes and duration values expected once we reached the actual duration value of our previous note.

PMOD Specification, Usage, High-Level Design, Key-Implementation, and Challenges

Our PMOD extension for **4 points** allowed for volume adjustment of our FPGA even outside of the song-playing states. To do this,



we used a rotary encoder as our device of choice to be able to increase and decrease the amplitude of our sample fed into the codec at any given time. Essentially, the rotary encoder includes two inputs (`pin_A` and `pin_B`) that go high 90 degrees out of phase and based on this logic, we can store the previous A signal and compare it to the current one. If they are different, then that means the rotary encoder has been rotated. We can then check if `pin_A`'s state is not equal to `pin_B`'s state, then the rotary encoder has been rotated clockwise and if they are the same, the rotary

encoder has been rotated counterclockwise. We can translate this logic to a "`cur_count`" number from 0-20 to determine how much we shift by (we translate this into our `scale_amplitude` which is simply `cur_count / 2` to prevent overshifting). We also prevent overflow/shifting by ensuring this number does not go below or above this range through manual checks and in our top module, shift our sample by $(10 - \text{scale_amplitude})$. Something to note is that we also needed to debounce these outputs to ensure our volume doesn't instantly go from 0-100 and vice-versa. Based on input from TAs and previous students, we believe that 4 points are warranted as the main issues in tackling this extension were those more related to the hardware aspects in making sure we reduced noise and debounced where needed. Our testing included realistic scenarios such as `pin_A` going low before `pin_B` and `pin_A` going high before `pin_B`. It also included the other way around to simulate CCW rotation.

Note Player Specification, Usage, High-Level Design, Key-Implementation, and Challenges

Our note_player still held most of its original logic due its main function which is to obtain our sample from our sine_rom using our provided input note and duration in order to send it elsewhere to be able to interpret sine waves. The harmonics incorporation included scaling our

stepsize which we scaled

depending on our

weight_meta which

consists of our 2 MSB

coming from our

meta_data (extracted from

song_reader which is

extracted from the

song_rom). This is only

considered if don't have a

weight of 2'b00 because

this case wouldn't add a

scale meaning our step

size remains the same.

Additionally, we

incorporated states that

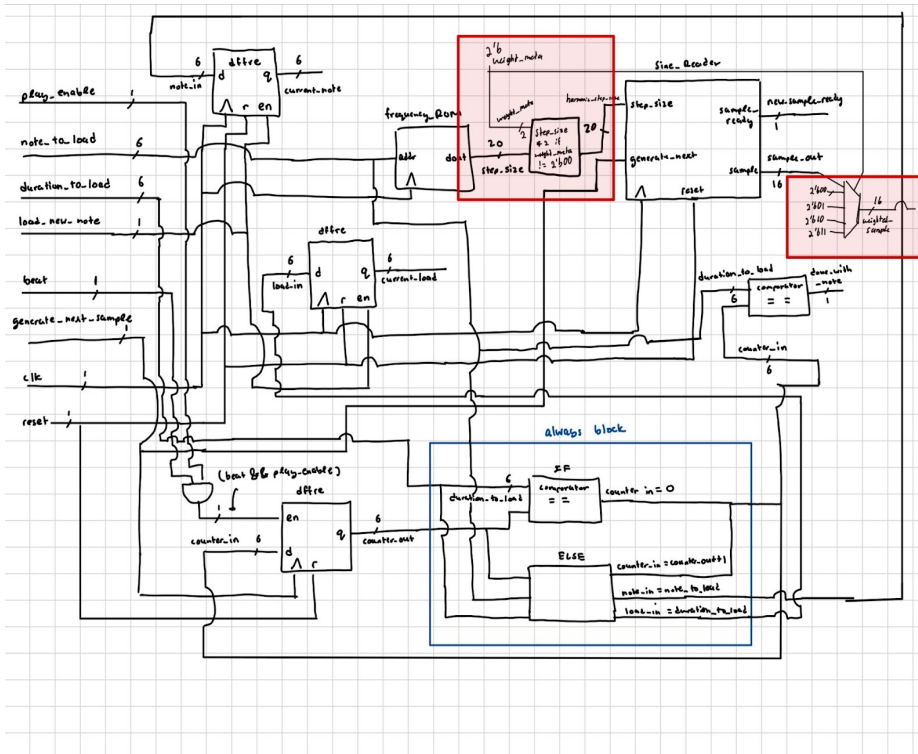
would allow us to scale

our output sample size.

This sample_size was

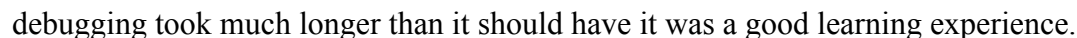
weighted depending on our

2 bits of meta_data and once we applied this weight considering our scaled stepsize we were able to output a harmonic as all it took was updating our inputs and outputs into its connected module music_player. The way we would get 3 notes still playing at the same time was because, in our music player, we would call note_player 3 times. This is how we manage to play 3 notes at the same time while using harmonics. The way this was tested was by checking whether our weight was changing our sample size and our final weighted sine waves. The general trend was that we would see more sine waves the more we added weight. Lastly, the overall challenge that we faced during this implementation was finding a way to keep my step size the same if I wasn't applying any weight through my meta_data. This was something that took a while to debug but found the waveform useful. This fix required a ternary operator allowing me to keep my step size the same if my weight was 2'b00.



Our pulse width modulation is what allowed us to incorporate changing the LEDs that would sync with our implementation of PMOD (rotary encoder) that we used to change the volume of our songs. The idea here was that we would keep a count that would count in intervals depending

on our duty cycle which would reset when the counter was ≥ 10000 . Furthermore, we used the opposite logic to add 1 and whenever our counter $\geq \text{duty} * 10000$ we would pulse our pwm_out high. This implementation is what allowed us to change the blink rate of our LED which would sync with our rotary encoder as we changed the volume. The higher the volume, the brighter the lights are, and vice versa. The way we tested this was by inputting different duty cycles and saw if my counter was changing in relation to the changes of the duty cycle (if we increased the duty cycle then the counter increases). Lastly, we made sure to check that pwm_out pulsed when my counter reset. Regarding the challenges we encountered, we found that whenever calling this in our top-level module we didn't send in clk_100 instead we input clk which wasn't even a variable. Although this



Our initial implementation of playback control consisted of using the first unassigned button on our FPGA board as a toggle switch that allows us to enter a second set of controls (see image below). When clicking the control set button we enter a PAUSE 2 state where we remap the play

and next buttons to rewind and fast-forward respectively (note: the reset button remains unchanged). From this state, clicking either rewind or fast-forward would change the state from PAUSE 2 to PLAY 2 in which all note durations and advance time durations would be scaled down by a half. While this implementation is more straightforward for fast-forward, rewinding would require not only the notes to be scaled down but also for the address we use to access our SONG_ROM to be decremented instead of incremented.

However, although we initially set out to implement playback control for a total of 4 points, we found serious challenges in being able to read the SONG_ROM backwards. This issue stemmed from the fact that when traversing our SONG_ROM in reverse, you encounter the advanced duration of the scheduled notes first which is in contrast to playing a song normally as you encounter the advanced duration last. This led us to attempt to find different ways to be able to retain the initial advance duration value we encounter and then use this value for the notes we schedule afterward. However, due to limited time and our music player crashing every time rewind was activated, we were unsuccessful in our initial goal for a full implementation of the playback extension. Nonetheless, the majority of our testing was done through our music_player test bench which involved simulating and rewinding a song expecting our song_rom input address to be decrement and the notes to be scheduled in reverse.

